

Background Information

Prepare `"orders_data.parquet"` so that it can be used to build a forecasting model.

- Clean the orders dataset as per the requirements specified in the Workbook.
- Save the updated file as `orders_data_clean.parquet`.

As a Data Engineer at an electronics e-commerce company, Voltmart, you have been requested by a peer Machine Learning team to clean the data containing the information about orders made last year. They are planning to further use this cleaned data to build a demand forecasting model. To achieve this, they have shared their requirements regarding the desired output table format.

An analyst shared a parquet file called `"orders_data.parquet"` for you to clean and preprocess.

You can see the dataset schema below along with the **cleaning requirements**:

orders_data.parquet			
column	data type	description	cleaning requirements
order_date	timestamp	Date and time when the order was made	Modify: Remove orders placed between 12am and 5am (inclusive); convert from timestamp to date
time_of_day	string	Period of the day when the order was made	New column containing (lower bound inclusive, upper bound exclusive): "morning" for orders placed 5-12am, "afternoon" for orders placed 12-6pm, and "evening" for 6-12pm
order_id	long	Order ID	N/A
product	string	Name of a product ordered	Remove rows containing "TV" as the company has stopped selling this product; ensure all values are lowercase
product_ean	double	Product ID	N/A
category	string	Broader category of a product	Ensure all values are lowercase
purchase_address	string	Address line where the order was made ("House	N/A

column	data type	description	cleaning requirements
		Street, City, State Zipcode")	
purchase_state	string	US State of the purchase address	<i>New column containing: the State that the purchase was ordered from</i>
quantity_ordered	long	Number of product units ordered	N/A
price_each	double	Price of a product unit	N/A
cost_price	double	Cost of production per product unit	N/A
turnover	double	Total amount paid for a product (quantity x price)	N/A
margin	double	Profit made by selling a product (turnover - cost)	N/A

```
In [6]: from pyspark.sql import (
        SparkSession,
        types,
        functions as F,
    )

    # Initiate a Spark session
    spark = (
        SparkSession
        .builder
        .appName('cleaning_orders_dataset_with_pyspark')
        .getOrCreate()
    )
```

Goal: Prepare and clean raw order data (orders_data.parquet) to make it ready for demand forecasting.

```
In [7]: # IMPORT DATA
        # Read data from the parquet file

        orders_data = spark.read.parquet('../data_raw/orders_data.parquet')
        orders_data.toPandas()
```

Out [7]:

	order_date	order_id	product	product_id	category	purchase_addr
0	2023-01-22 21:25:00	141234	iPhone	5.638009e+12	Vêtements	944 Walnut St, Boston, MA 02111
1	2023-01-28 14:15:00	141235	Lightning Charging Cable	5.563320e+12	Alimentation	185 Maple St, Portland, OR 97201
2	2023-01-17 13:33:00	141236	Wired Headphones	2.113973e+12	Vêtements	538 Adams St, San Francisco, CA 94102
3	2023-01-05 20:33:00	141237	27in FHD Monitor	3.069157e+12	Sports	738 10th St, Los Angeles, CA 90015
4	2023-01-25 11:59:00	141238	Wired Headphones	9.692681e+12	Électronique	387 10th St, Austin, TX 78701
...	...	...	...	...	...	...
185945	2023-12-11 20:58:00	319666	Lightning Charging Cable	6.545974e+12	Électronique	14 Madison St, San Francisco, CA 94102
185946	2023-12-01 12:01:00	319667	AA Batteries (4-pack)	5.352480e+12	Électronique	549 Willow St, Los Angeles, CA 90015
185947	2023-12-09 06:43:00	319668	Vareebadd Phone	2.674213e+12	Alimentation	273 Wilsor St, Seattle, WA 98101
185948	2023-12-03 10:39:00	319669	Wired Headphones	5.216304e+12	Alimentation	778 River St, Dallas, TX 75201
185949	2023-12-21 21:45:00	319670	Bose SoundSport Headphones	8.081038e+12	Électronique	747 Chestnut St, Los Angeles, CA 90015

185950 rows x 11 columns

1. Remove Unnecessary Orders:

- Orders placed between 12 AM - 5 AM were removed because they might be anomalies or system-generated.
- Filter out these transactions.

2. Create a "Time of Day" Column:

- Helps analyze demand patterns at different times of the day.
- Categorized purchases into morning, afternoon, and evening.

In [8]: # DATA CLEANING AND PREPROCESSING

```
orders_data = (
    orders_data
    # Create a new column time_of_day
    .withColumn(
        'time_of_day',
        # When/otherwise (similar to case/when/else) statements extracting h
        F.when((F.hour('order_date') >= 0) & (F.hour('order_date') <= 5), 'r
        .when((F.hour('order_date') >= 6) & (F.hour('order_date') <= 11), '
        .when((F.hour('order_date') >= 12) & (F.hour('order_date') <= 17),
        .when((F.hour('order_date') >= 18) & (F.hour('order_date') <= 23),
        # You can keep the otherwise statement as None to validate whether t
        .otherwise(None)
    )
    # Filter by time of day
    .filter(
        F.col('time_of_day') != 'night'
    )
    # Cast order_date to date as it is originally a timestamp
    .withColumn(
        'order_date',
        F.col('order_date').cast(types.DateType())
    )
)
```

3. Standardize Product Names:

- Some product names were inconsistent (e.g., uppercase/lowercase mix).
- Convert all product names to lowercase.

4. Remove Discontinued Products

- The company stopped selling TVs, so orders for TVs were removed.
- Filter out rows where the product name contains "TV".

```
In [9]: orders_data = (
    orders_data
    # Make product and category columns lowercase
    .withColumn(
        'product',
        F.lower('product')
    )
    .withColumn(
        'category',
        F.lower('category')
    )
    # Remove rows where product column contains "tv" (as you have already ma
    .filter(
        ~F.col('product').contains('tv')
    )
)
```

```
In [10]: orders_data = (
    orders_data
    # First you split the purchase address by space (" ")
    .withColumn(
        'address_split',
        F.split('purchase_address', ' ')
    )
    # If you look at the address lines, you can see that the state abbreviat
    .withColumn(
        'purchase_state',
        # F.col('address_split').getItem(F.size('address_split') - 2)
        F.col('address_split')[F.size('address_split') - 2]
    )
    # Dropping address_split columns as it is a temporary technical column
    .drop('address_split')
)

# Use distinct and count to calculate the number of unique values
n_states = (
    orders_data
    .select('purchase_state')
    .distinct()
    .count()
)
```

6. Save cleaned data:

- the dataset was saved as orders_data_clean.parquet

```
In [11]: # EXPORT
# Export the resulting table to parquet format with the new name
(
    orders_data
    .write
    .parquet(
        './data_cleaned/orders_data.parquet_cleaned',
        mode='overwrite',
    )
)
```

```
In [12]: # jupyter nbconvert --to HTML orders-dataset-cleaning.ipynb
```